

Robustness of Bayesian Neural Networks Design Choices Against Weight and Bias Perturbation Induced by Single Event Upsets (SEUs)

Revalda Putawara^a, Dalila O’Grady^b, James Pope^c

Intelligence Systems Laboratory, University of Bristol, UK
ey24622@bristol.ac.uk, mv22726@bristol.ac.uk, james.pope@bristol.ac.uk

Keywords: Bayesian Neural Networks, Robustness, Single Event Upsets, Prior Distributions

Abstract: Bayesian Neural Networks (BNNs) model uncertainty by treating weights and biases as probability distributions, making them attractive for deployment in environments where uncertainty can accommodate unexpected perturbations, leading to improved robustness. A key fault source is Single Event Upsets (SEUs), caused by radiation that can occur in harsh environments. Although robustness to input perturbations has been widely studied for both deterministic and Bayesian CNNs, weight and bias perturbations is not as widely explored, particularly for Bayesian CNN models. The growing use of AI in space missions and other fault-prone environments heightens the urgency in understanding the robustness of BNNs to weight/bias perturbations like SEUs. To address this gap, we offer the solution of introducing Bayesian Neural Networks in harsh environments by simulating it under SEU injections and exploring which design gives better robustness. In this study, deterministic and Bayesian CNN models were subjected to SEU injections across layers, parameters, bit positions, and neuron locations. Robustness was quantified using a new Aggregate Robustness Index (ARIn), combining accuracy and softmax differences. Results show deterministic CNNs achieved higher baseline accuracy (97.55% vs. 82.85%), but BNNs were more robust (ARIn 0.0779 vs. 0.1909). Dropout yielded the largest robustness gain, with Gaussian priors outperforming Laplace and Uniform. SEUs on variance parameters were more harmful than on means, and fully connected layers proved most vulnerable. Sigmoid was the most robust activation despite an accuracy trade-off. Overall, BNNs provide more stable prediction confidence, making them suitable for safety-critical applications such as medical imaging and ensemble systems.

1 Introduction

Machine learning (ML) models are widely used in our daily life. Most of ML systems can be found close to us, like a recommender system, large language model or even autonomous vehicle that utilizes Convolutional Neural Network (CNN) models to steer clear from obstacles. While most of those systems are deployed in a normal environment, some of ML models are also deployed in harsh environments with its own specific designated purpose.

A representative example is CloudScout (Giuffrida et al., 2020), a CNN-based system onboard the ESA PhiSat-1 mission that performs on-board cloud detection on hyperspectral images, transmitting only non-cloudy data to the ground, reducing bandwidth usage.

Machine learning systems deployed in harsh environments are exposed to risks during its inference, and a common term to associate this performance with Deep Neural Networks is *robustness* (Shafique et al., 2021). Robustness are categorized into two sub-properties, security and reliability. While the security aspect of DNNs focuses on preventing information inside the network to be stolen or protecting the system to get an incorrect input (adversarial attack), the reliability aspect of robustness focuses on keeping the model to display a consistent output, parameter, or behaviour, despite changes of environmental condition or fabrication.

Being deployed in the space, a harsh environment, we can assume that machine learning systems like CloudScout is prone to risks that exists due to being deployed outside of earth surface. One of the risks that could hamper the performance of machine learning models that are deployed in harsh environments is single event effects (SEEs).

One example of SEE that can happen to ma-

^a  <https://orcid.org/0009-0008-7938-185X>

^b  <https://orcid.org/0000-0002-7982-2917>

^c  <https://orcid.org/0000-0003-2656-363X>

chine learning models deployed in a harsh environment is Single Event Upsets (SEUs). SEUs are a temporary change of memory or control bit (Petersen, 2011). One phenomenon that shows the characteristic of SEUs is the incident of bitflips.

Machine learning with a good quality of robustness, or Robust ML Systems, can mitigate the impact of SEUs/bitflips, rendering the output of the model to be consistent. In this research, we are motivated to understand the impact of SEUs/bitflips to weight/bias perturbation in Bayesian Neural Networks.

This research is built upon work done in (Dennis and Pope, 2025) that reviewed CNN design choices to develop robust ML models. This research is also motivated by previous research that has proved that Bayesian Neural Networks are more robust against adversarial attacks (security aspect of robustness). Although there are already a significant number of studies that addressed those two aforementioned fields, **there has been little to no investigation into the robustness of Bayesian Neural Networks against weight/bias perturbation** (reliability aspect of robustness). This dissertation seeks to address that gap.

In this research, we will show multiple Bayesian Neural Network design choice and compare its performance measure. Subsequently, we will simulate Single Event Upset (SEUs) with bitflips across all Bayesian Neural Network design alternatives and measure its performance measures afterwards. Robustness is then measured from the difference on the result of the model, before and after the model is perturbed with SEUs.

We comprehensively compared deterministic and Bayesian CNN models' performance across 4 model variations, 7 activation functions, and 3 prior distributions. In this research, we also proposed a new robustness measure called the Aggregate Robustness Index (ARIn), which combines accuracy change and softmax deviation. Overall, this study show whether Bayesian CNN model can achieve better robustness compared to deterministic CNN model or not. It also shows which model design that is more robust.

This study provides the evaluation of BNN robustness under SEUs, which addresses current research gaps in prior work that has focused primarily on CNN robustness under SEUs and Bayesian CNN robustness under adversarial attacks. The paper makes the following contributions:

- We demonstrate a range of Bayesian Neural Network design choices that are robust against perturbation of the model's weight and biases.
- We provide detailed analyse of the effects of SEUs across different attack variables, including layer type, weight versus bias, mean versus variance

parameters, and bit index along with its original value.

- Propose a novel measure of model robustness for SEUs. We show that the measure is capable of detecting critical aspects of models under SEUs attacks.

2 Background

2.1 Robust Machine Learning Systems

2.1.1 Robustness against Adversarial Attack

Before we take a look into robustness of neural networks against SEUs/bitflips, we will see what are the defenses against adversarial attack applied in recent research, especially for Bayesian Neural Networks.

BNNs have been reported to mitigate gradient-based input attacks, e.g., Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD), due to posterior averaging (Carbone et al., 2020). There are also methods that employ multi-task loss combining unsupervised feature scattering and supervised margin loss to generate diverse and stronger adversarial examples for BNN training (Chen et al., 2022). **Spectral Expectation Bound Regularization (SEBR)** (Zhang et al., 2021) also increases robustness against gradient-based attacks by regularizing the spectral norm of weights. Contradicting work includes (Blaas, 2021), which shows that a more accurate posterior inference (e.g., Hamiltonian Monte Carlo, advanced VI) does not guarantee robustness.

2.1.2 Robustness against Weight Perturbation

Single Event Upset Induced Parameter Perturbation (SIPP) (Yan et al., 2020) shows that SEUs in memory or parameters can drastically degrade accuracy. Proposed defense in this work includes Triple Modular Redundancy (TMR), Error-Correcting Codes (ECC), and Selective Protection on both.

Model-intrinsic robustness is explored in (Dennis and Pope, 2025), where Weighted Gaussian (WG) and Rectified Weighted Gaussian (RWG) activation functions, smart pooling, dropout, and weight decay show improvements in deterministic CNNs under SEUs.

While both of the works mentioned previously take an interest in the robustness of deterministic neural networks, especially variants of convolutional neural networks, against weight/bias perturbation, **no significant research is found on observing Bayesian Neural Networks (BNNs) robustness**

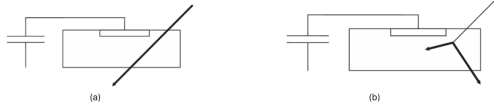
against weight perturbation yet. This area of research is what this paper aims to look into.

2.2 Single Event Upsets

Single Event Effects (SEEs), as described in (Petersen, 2011), happened through the action of a single ionizing particle as it penetrates sensitive nodes inside an electronic device, illustrated in Figure 1. It has been linked to a real-world system faults, for example, the 2008 Qantas A330 altitude change incident is suspected to be caused by a possible SEE (Australian Transport Safety Bureau, 2009). Some variants of SEEs are Single Event Upsets (SEUs), Multibit Upsets (MBUs), and Single Event Burnouts (SEBs).

SEUs have been simulated either at the software level, by forcing bit values in CNN weights (Bosio et al., 2019), or at the hardware level, by injecting transient bit-flips into GPU registers (Hari et al., 2017). Both approaches show that SEUs can cause incorrect outputs in deep networks, reducing reliability in safety-critical tasks. This study follows (Hari et al., 2017), introducing bit-flips to model weights and biases **when it is doing inference on a new data**.

Figure 1: Ionization path due to direct passage of heavy iron (a), and due to proton reaction in a device (b) as shown in (Petersen, 2011)



The IEEE-754 standard ((IEEE Computer Society, 2019)) defines binary (32, 64, 128 bits) and decimal (64, 128 bits) floating-point (FP) formats. Modern CPUs and frameworks (e.g. PyTorch) support binary machine numbers (FP32, FP64) for efficiency despite limited precision. Since Pyro is built on PyTorch, this research uses FP32, making it important to understand floating-point representation for analyzing the effects of Single Event Upsets (SEUs) on weight and bias parameters stored in FP32.

2.3 Bayesian Neural Networks

In this section, we will take a look at the concept of Bayesian Neural Networks. This research assumes that the reader is familiar with the basics of Neural Networks, including the concept of feed-forward and backpropagation. The concept of Bayesian Neural Networks (BNNs) originated in the early 1990s. (MacKay, 1992) introduced a Bayesian framework for neural learning by treating weights and biases

as probabilistic variables, addressing uncertainty and overfitting.

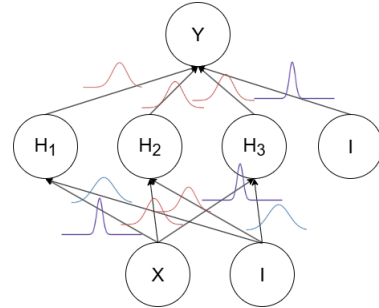
Equation 1 - 2 express the core of Bayesian learning. The marginal likelihood (Equation 1) integrates over all weights configurations, embodying Occam's razor to balance model fit and complexity. Bayes' rule (Equation 2) updates prior beliefs with data, yielding weight distributions that capture model uncertainty.

$$P(\mathcal{D} | \mathcal{M}) = \int P(\mathcal{D} | \mathbf{w}, \mathcal{M}) P(\mathbf{w} | \mathcal{M}) d\mathbf{w} \quad (1)$$

$$p(\mathbf{w} | \mathcal{D}) \propto P(\mathcal{D} | \mathbf{w}) p(\mathbf{w}) \quad (2)$$

In (MacKay, 1992), approximations are made to the posterior by using Laplace approximation and use Hessian of the loss to estimate posterior curvature. The term Bayesian Neural Network first coined in (Neal, 1996). In that work, Radford M. Neal take the posterior distribution approximation to a new direction, by using Markov Chain Monte Carlo (MCMC) to not just do evidence maximization, but also sampling from the full posterior over weights, to approximate $p(\mathbf{w} | \mathcal{D})$. Illustration on how Bayesian Neural Networks differ to the Deterministic Neural Networks, in which each weight is assigned a distribution, is shown in Figure 2.

Figure 2: Illustration of Bayesian Neural Networks, as shown in (Blundell et al., 2015)



3 Methodology

3.1 Dataset

In this research, we are using an image dataset called "Ships in Satellite Imagery" (Hammell, 2018), or "Shipsnet" in short. The dataset is satellite imagery captured from a commercial company called Planet. Those satellite images are taken from over the San Francisco Bay and San Pedro Bay areas of California. Initially, the pictures are orthorectified 3-meter pixel size, but for this task in detecting ships, 4000 80 x 80

RGB images are derived from it, labeled with either a "ship" (1) or "no-ship" (0) classification.

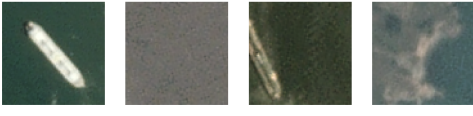


Figure 3: Dataset

The image is resized into 64 x 64 pixels, its pixel values are scaled, and then standardized, so that its value ranges from -1 to 1 and is centered around 0.

3.2 Model architecture

Baseline. The deterministic CNN model that we will set as a benchmark for this research is designed for binary image classification, as the class distribution dictated in subsection 3.1. We will utilize 2 convolutional layers (namely `conv1` and `conv2`) with each followed by designated activation function and max-pooling (`torch.nn.MaxPool2d`). The output then flattened to be processed in a fully connected layer that outputs two class probability. In total, we will have 52,162 trainable weights and biases.

Training. The model is trained on Shippnet dataset with 80/20 train-test split. The model will be trained on the dataset of batch size 16 in 20 epochs, keeping the model with the best test accuracy. Adam is used to minimize the loss function with learning rate of 10^{-3} .

Bayesian extension. The Bayesian CNN model will follow the architecture of the deterministic CNN model, with a few differences. First, in this model, all weights and biases are modelled as random variable drawn from a prior distribution, which either Gaussian, Laplace, or Uniform prior. This variation will allow us to study how the tail behavior and spread of each prior influence robustness.

Bayesian model training. Our Bayesian CNN model will use negative Evidence Lower Bound (ELBO) loss. Given the dataset D with observed inputs and labels, the ELBO loss aims to maximize the expected log-likelihood of the data from the sampled network weights while minimizing the KL divergence between the learned variational distribution $q(W)$ and the prior $p(W)$, trying to balance the data fit and regularization. ELBO is notated as follows:

$$\text{ELBO} \equiv \mathbb{E}_{q_\phi(W)} [\log p(D | W)] - \text{KL}(q_\phi(W) \| p(W)) \quad (3)$$

Bayesian Inference of these weights and biases during training will utilize Stochastic Variational Inference (SVI), with mean-field assumption. All weights and biases are assumed to be independent.

Inference procedure. Inference in Bayesian CNN models in this research is different from what is usually done in deterministic CNN models. We will use Monte Carlo Sampling in doing inference, with number of Samples $S = 10$. Each iteration samples new weights ($W^{(s)}$) from a variational posterior family. Logits are averaged before applying argmax, reducing prediction variance. This mechanism shows the aggregation of Bayesian predictive uncertainty, which can not be found in the deterministic CNN.

3.3 Model Variations

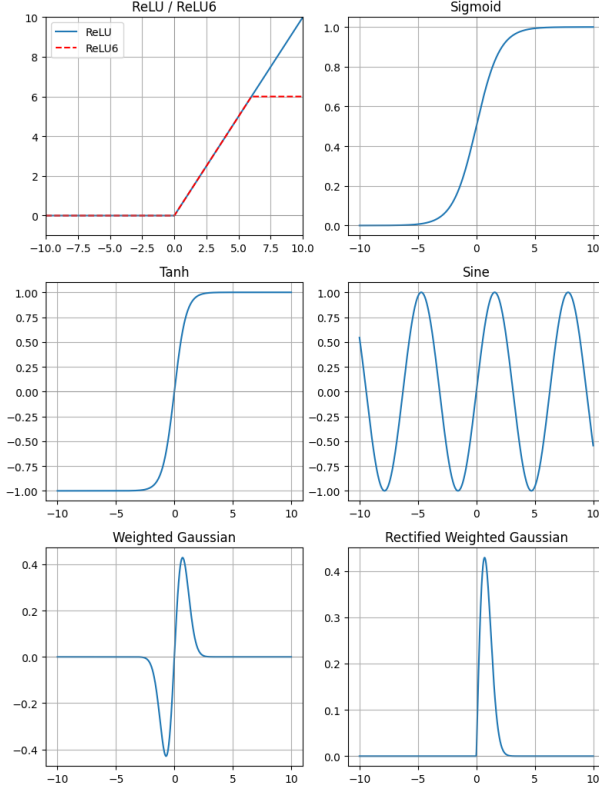
In this study, we will utilize 7 different activation functions to compare their robustness. Some of these are widely known activation functions like ReLU, Sigmoid, Tanh, and Sine. We also want to test the Weighted Gaussian (WG) and Rectified Weighted Gaussian introduced in (Dennis and Pope, 2025) in this experiment, which are shown in Equation (4) and Equation (5). As previous research shows that ReLU is the least robust activation function (Dennis and Pope, 2025), we also want to look at a different version of it called ReLU6, that clips ReLU to have a maximum value of 6.

$$\text{WG}(x) = xe^{-x^2} \quad (4) \quad \text{RWG}(x) = \max(0, xe^{-x^2}) \quad (5)$$

To observe different model architecture design and their impact to the robustness of our model, we will explore a few options that we can choose to apply to that model, on top of our base model. The different model architecture designs are coded as follows:

1. Model 0: The base model.
2. Model 1: Instead of using maxpooling, the convolutional layer will be followed by smartpooling. This method is inspired from the work in (Santos et al., 2019), that demonstrates that 16% robustness is achieved in ReLU-based deterministic CNNs equipped with SmartPooling. In Smartpooling, if a value exceeds a threshold $\theta = 10x$, the value will be discarded and the second highest value is chosen.
3. Model 2: Dropout layer is applied after conv2 layer during training. Dropout layer can benefit from mitigating the SEU-induced performance drop (Dennis and Pope, 2025) by deactivating random neurons during training, with $p = 0$ (off state) and $p = 0.5$ (on state) in this experiment.
4. Model 3: Weight decay is implemented to the Adam optimizer with $\lambda = 10^{-4}$. Weight decay is a regularization technique that allows complex models to be trained on data sets of limited size without severe over-fitting (Bishop, 2007).

Figure 4: Activation Functions Plot



While the deterministic CNN model have 28 model variations (7 activation functions and 4 architecture designs), the Bayesian CNN model that relies on sampling weight and bias from prior distribution will also be varied by the 3 different prior distribution (Gaussian, Laplace, and Uniform). This makes the Bayesian CNN model to have 84 model variations.

3.4 Simulated SEU Process

In this research, we are using Floating Point 32 (FP32) to represent numbers in our machine. SEU injection will do a bitflip in one of the bit in FP32, changing the value from 0 to 1 or vice versa. Illustration of an SEU injection in a deterministic CNN model is as follows:

- Layers are denoted as m and each layer has N_m output neurons.
- The neuron position under attack is denoted as $\text{pos} \in \{f, \ell\}$, where f corresponds to the first neuron and ℓ to the last neuron, defined as:

$$n_m^f = 1, \quad n_m^\ell = N_m.$$

- Parameter type is $p \in \{w, b\}$ for weight or bias.

- The scalar value of weight or bias is represented as $w_{m,n,p}$.
- A single attacked bit index i follows the IEEE-754 32-bit floating-point representation, $i \in I$

For a selected layer m , parameter type p , neuron position pos , and bit index i , the SEU targets the scalar parameter belonging to neuron $n = n_m^{\text{pos}}$. Its bit representation is:

$$\text{bits}(w_{m,n,p}) = (b_0, \dots, b_i, \dots, b_{31}), \quad b_i \in \{0, 1\}.$$

Let bit $j \in \{0, 1, \dots, 31\}$ be all bit value after bit i is attacked by SEU, according to the following rule:

$$b'_j = \begin{cases} 1, & j = i \text{ and } b_i = 0, \\ 0, & j = i \text{ and } b_i = 1, \\ b_j, & j \neq i. \end{cases} \quad (6)$$

The perturbed (post-SEU) scalar value $\tilde{w}_{m,n,p}$ is then defined as FP32 representation of its bits:

$$\tilde{w}_{m,n,p} = \text{FP32}(b'_0, b'_1, \dots, b'_{31}) \quad (7)$$

For our deterministic CNN models, we will do 2,352 SEU injections. This SEU injections are the combination of 28 different models that we have (4 model variations and 7 activation functions), with each model get 84 different SEU injections, where each attack is specified by $m \in \{\text{conv1}, \text{conv2}, \text{fc1}\}$, $\text{pos} \in \{f, \ell\}$, $p \in \{w, b\}$, and $i \in I = \{0, 1, 3, 6, 10, 15, 21\}$.

In our Bayesian CNN models, each parameter $w_{m,n,p}$ is treated as a random variable drawn from a variational posterior distribution. Let's take an example for a draw in a Gaussian distribution for Monte Carlo sample $s \in S$:

$$w_{m,n,p}^{(s)} \sim q_\phi(w_{m,n,p}) = \mathcal{N}(\mu_{m,n,p}, \sigma_{m,n,p}^2).$$

An SEU attack is applied before the weight/bias is sampled. Instead of applying SEU to $w_{m,n,p}^{(s)}$, it is applied to either $\mu_{m,n,p}$ (prior center parameter) or $\sigma_{m,n,p}^2$ (prior spread parameter). The SEU attack procedure follows the procedure that has been explained earlier in this subsection.

For our Bayesian CNN models, we will do 15,456 SEU injections. 84 different models will be attacked with 84 (7×12) attacks for the prior center and 72 (6×12) attacks for the prior spread, for a total of 13,104 SEU injections. The combination of attacks is similar to what we have in the deterministic CNN models. The only difference is for the prior spread parameter, the bit index attacked is limited to $i \in I = \{1, 3, 6, 10, 15, 21\}$. This is due to hitting the bit index

0 for prior spread will change the prior spread parameter sign from + to -, which is not valid to represent the spread parameter of our prior.

The total SEU injection attacks that we will do in this study is 15,456 injections.

3.5 Robustness Measure

To evaluate how well a neural network withstands adversarial attack or weight/bias perturbation, we need a quantitative metric that can measure its robustness. This measure will help us in comparing the variation of models or strategies with each other and decide which ones are the most robust against perturbations.

The first metric that we can use to measure robustness is accuracy delta/accuracy difference. It is the most commonly used measure of robustness in this field of research and can be found in (Dennis and Pope, 2025), (Feng et al., 2024), and (Pang et al., 2021). It measures the performance difference before/after perturbation. Smaller values are perceived to show a better robustness.

Accuracy of the model after the SEU injection does not necessarily lower than the accuracy before the SEU injection is done, and we want to look both for positive or negative change of accuracy and penalize them equally, without cancelling each other. To do that, we will modify the formula of Accuracy Delta to be called Absolute Accuracy Delta (AAD) as follows:

$$\text{Absolute Accuracy Difference} = |\text{Accuracy}_{\text{after}} - \text{Accuracy}_{\text{before}}| \quad (8)$$

Softmax Difference (Carbone et al., 2020) captures prediction confidence change. Robustness is $1 - \text{softmax difference}$. Previous work (Su et al., 2018) highlights the trade-off between accuracy and robustness.

The metric was originally defined for input perturbation, comparing the network output on a clean test point against its adversarially crafted counterpart under a single posterior. Our perturbation is applied to the weights rather than the input, so we adapt it accordingly: the test input is held fixed and the two expectations are taken under the original variational posterior $q_\phi(\mathbf{w})$ and the SEU-perturbed variational posterior $\tilde{q}_\phi(\mathbf{w})$ of Equation (7).

Softmax Difference =

$$\frac{1}{N} \sum_{j=1}^N \left| \langle f(\mathbf{x}_j, \mathbf{w}) \rangle_{q_\phi(\mathbf{w})} - \langle f(\mathbf{x}_j, \mathbf{w}) \rangle_{\tilde{q}_\phi(\mathbf{w})} \right|_\infty \quad (9)$$

Here $\langle \cdot \rangle$ denotes the Monte Carlo average of the softmax output over S weight samples drawn from the

respective posterior, and $|\cdot|_\infty$ the maximum absolute deviation across class probabilities, averaged over the N test points. Our research questions rely on comparing multiple model variations and deciding which one is the best based on the given result. While both robustness measures are useful in achieving our goals on its own, interpreting both of the robustness measure at the same time may result in confusion, e.g., one model might have a low absolute accuracy difference, but a high softmax difference. To make the model comparison easier, we devise a new aggregate robustness measure called the Aggregate Robustness Index (ARIn), which takes Equation (8) and Equation (9).

$$\text{ARIn} = \sqrt{\frac{(\text{Absolute Accuracy Difference})^2 + (\text{Softmax Difference})^2}{2}} \quad (10)$$

4 Results

4.1 Training Result

Model Variant	DNN	BNN
Model 0	.9757	.8288
Model 1	.9752	.8332
Model 2	.9763	.8276
Model 3	.9748	.8243
Average	.9755	.8285

Table 1: Test accuracy of Bayesian and Deterministic CNN Models grouped by model variants

Based on our findings in this chapter, **Bayesian CNN model is showing an inferior performance in test accuracy** compared to the deterministic CNN model. This observation is consistent across all activation functions, model variants, and Bayesian model activation function. The average accuracy of test data that can be achieved by the deterministic CNN model is 97.55%, while the Bayesian CNN model can only achieve 82.85%. it's suspected that the performance difference is inherent to whether we are implementing Bayesian or deterministic CNN model. Here are some factors that can possibly result in this observation:

1. All weights and bias in the Bayesian CNN neural network are initialized with the same prior distribution parameter (a and b).
2. Optimization of Bayesian Neural Networks with ELBO loss is harder compared to cross-entropy

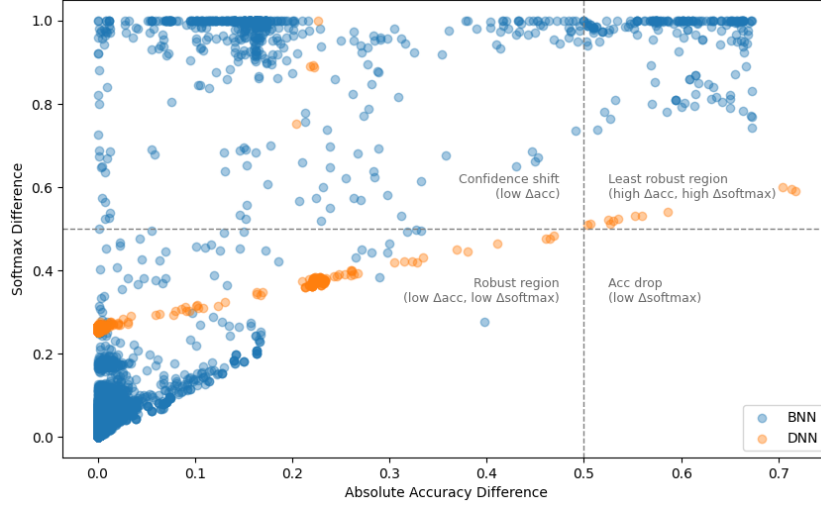


Figure 5: Robustness of Deterministic and Bayesian CNN model

in deterministic CNN model. KL divergence in ELBO loss can pull the variational posterior distribution away from the optimal point to stay closer to the prior (posterior collapse).

In the subsequent subchapters that measure robustness of deterministic CNN models and Bayesian CNN models, most table will provide the aggregate value of SEU robustness measure by grouping the experiment result based on several criteria and then averaging them. For simplicity, we denote these aggregated metrics as Average Absolute Accuracy Difference (AAAD) and Average Softmax Difference (ASD). Aggregated Robustness Index (ARIn) will be calculated based on these two values. The lowest value for each column will be typed in bold.

4.2 Robustness by model type (BNN vs DNN)

Model Type	AAAD	ASD	ARIn
DNN	.0149	.2696	.1909
BNN	.0244	.1074	.0779

Table 2: Summary of average absolute accuracy difference and average softmax difference Deterministic vs Bayesian CNN

4.2.1 Robustness of Deterministic CNN models

From Figure 5, we can infer that the impact of SEU on Absolute Accuracy Difference and Softmax Difference exhibits an approximately linear relationship, showing a clear correlation between the two perfor-

mance metrics. This observation is similar from what has been observed from (Carbone et al., 2020), in which the author spotted a trade-off between pure accuracy and robustness ($1 - \text{softmax difference}$) for high performing deterministic networks against adversarial attack (FGSM).

4.2.2 Robustness of Bayesian CNN models

Contrary to the Deterministic CNN model SEU injection result that has a clear correlation between the two performance metric in a linear straight line, the performance shown by the Bayesian CNN model does not exhibit a same pattern. In Bayesian CNN Model SEU Injection from the figure above, some noticeable points make up a linear correlation of absolute accuracy difference and softmax difference, but there are also observations that does not support this relationship. Some SEU Injection results show high softmax difference, while still keeping the absolute accuracy difference low.

4.2.3 Robustness of Deterministic CNN models vs Bayesian CNN Models

Based on Table 2, it is observed that the deterministic CNN model is showing more robustness against SEU injection in absolute accuracy difference. It shows that the average absolute accuracy difference if a deterministic CNN model is attacked by SEU injection is 1.49%, while the Bayesian CNN model is expected to have an absolute accuracy difference of 2.44%.

This phenomenon can be triggered by the way that Bayesian Neural Network is sampling the weight/bias distribution parameter from the guide. In deterministic CNN model, the SEU only change one fixed

weight/bias. In Bayesian CNN Model, as the distribution parameter is perturbed by the SEU, all Monte Carlo (MC) samples are drawn from the same perturbed distribution parameter. This makes every MC sample to be perturbed in the same way. This, will in turn disturb the average logits that is needed to classify the data with softmax. The resulting condition would be the data that is more prone to misclassification due to disturbed sampled weight and bias.

On the other hand, in terms of average softmax difference, Bayesian CNN model can be seen to demonstrate a better robustness against SEU injection. Softmax difference, which indicates the change in the predicted class probability, is observed to be 0.2696 on the Deterministic CNN Model, and 0.1074 on the Bayesian CNN Model. This means that Bayesian CNN Model has a smaller shift in output probabilities compared to the Deterministic one.

The observed condition can happen due to how logits are calculated after MC sampling. As we run over the network 10 times, or as much as the number of samples is determined, logit shifts from the first run might be diluted by the subsequent run, as we are taking the average of all 10 runs to be the input of the softmax, that will classify our data. Although the chance of misclassifying some data sample is increased, Bayesian CNN model offer a more stable prediction probability against SEU.

To conclude our comparison, we employ the Aggregate Robustness Measure (ARIn) defined in Equation (10). Based on this metric, we find that **Bayesian Neural Network exhibits greater robustness than the Deterministic Neural Network**, as evidenced by its smaller ARIn value (0.0779 compared to 0.1909 for the latter). In the applications where class confidence or predictive probability constitutes a critical factor, the Bayesian CNN model is therefore more suitable to use compared to the deterministic CNN model. Some applications can include medical imaging (e.g. If the model uncertainty exceeds the threshold, the case is "rejected" or flagged for human evaluation (Ferrante et al., 2024)) and ensemble modeling (in which multiple models are combined to produce aggregate predictions (Barber and Bishop, 1998)).

4.3 Robustness by model variation

In Table 3, each row represents model variations explained in Subsection 3.3.

4.3.1 Robustness of Deterministic CNN models

Table 3 summarizes these results by averaging the Absolute Accuracy Difference (AAAD) and Softmax Difference (ASD) across model variants. Based

	AAAD		ASD		ARIn	
	DNN	BNN	DNN	BNN	DNN	BNN
0	.0145	.0243	.2694	.1106	.1908	.0800
1	.0142	.0254	.2699	.1109	.1911	.0805
2	.0157	.0221	.2702	.0944	.1914	.0685
3	.0152	.0257	.2687	.1138	.1903	.0825

Table 3: Robustness of Deterministic and Bayesian CNN model by model variant

on the Aggregate Robustness Index (ARIn), **Model 3 (trained with weight decay) demonstrates the strongest robustness**. Although it exhibits slightly worse AAAD compared to the base model (Model 0), it achieves a noticeably lower ASD. This is consistent with the expectation that weight decay reduces the magnitude of the weights, leading to a smaller logits and hence smaller softmax differences.

In comparison to prior work (Dennis and Pope, 2025), our findings diverge. While it suggests that a variant equivalent to our Model 2 achieved the best AAAD, our experiments indicate that Model 2 has the least robust AAAD, while Model 1 (with smartpooling) performs best in AAAD.

4.3.2 Robustness of Bayesian CNN models

As shown in Table 3, **Model 2 (trained with dropout) is the most robust among the Bayesian variants**, exhibiting the lowest values for both AAAD and ASD. Although AAAD is generally similar across Bayesian variants, the reduced ASD of Model 2 makes its particularly notable. This outcome aligns with findings in (Dennis and Pope, 2025), despite differences in experimental setup.

4.3.3 On the robustness of Bayesian CNN model trained with dropout layer

The significant robustness of Model 2 compared to other model variants in the Bayesian setting can be attributed by the nature of Dropout layers that demonstrate redundancy during training. As some neurons are inactive during training, our neural network can not just too reliant on any single neuron to carry critical information/infer a feature. In our Bayesian CNN model without dropout layer, there might be a chance that a SEU/bitflips can hit a dominant neuron and shift our prediction significantly. With a regularized model with dropout, the resulting model has no dominant neuron, so during inference, other neurons are as contributing as others, resulting in less shift in prediction if one of the neurons is being hit by bitflip.

4.4 Robustness prior and prior parameter

	AAAD		ASD		ARIn	
	C	S	C	S	C	S
G	.0177	.0293	.0794	.1281	.0575	.0929
L	.0193	.0287	.0823	.1306	.0598	.0946
U	.0231	.0303	.1054	.1277	.0763	.0928

Table 4: Robustness of Bayesian CNN model by prior and prior parameter

In Table 4, each row represents each prior distribution; which are Gaussian (G), Laplace (L), and Uniform (U). For each performance measure, we varied the SEU injection by the Center (C) and Scale (S) prior parameter.

4.4.1 Which prior is the most robust?

For the absolute accuracy difference, both Gaussian and Laplace prior have a similar result on the center and spread, with the Uniform distribution is the worst among all prior distributions. In terms of softmax difference, the Gaussian and Laplace distribution also performs similarly on the prior center and are both better compared to the Uniform distribution. A different observation can be found on the spread, which shows the Uniform and Gaussian perform similarly and better compared to the Laplace distribution. Overall, we can conclude that the **Gaussian prior is the most robust prior** among all, with Laplace prior shown a better accuracy robustness on prior spread attack, and the uniform distribution shown a better softmax robustness on prior spread attack.

4.4.2 Gaussian prior vs Laplace prior

Gaussian priors have lighter tails, making them more sensitive to mean or scale shifts that move samples across decision boundaries and increase prediction changes. Laplace priors, with heavier tails, are less affected by spread perturbations since their distributions are already wide, Uniform priors perform worse because their flat density causes any shift to affect many samples simultaneously, degrading accuracy.

4.4.3 Uniform prior robustness in softmax difference against the spread SEU injection

Since changing the scale parameter for the Gaussian and Laplace prior affect the peak height and decay rate in the tails, if we do Monte Carlo sampling for weight/bias sampling, the difference of sample values is expected to be higher compared to the Uniform

prior that have a flat PDF. This is due to the fact that the high-density center is now less dense and mass shifts into the tails, which will amplify the variability in logits. The uniform distribution can yield a better softmax difference, due to the fact that the each value of weight/bias have a similar probability to be sampled. A slight change in Uniform’s scale parameter, for a latent weight x , will result in a smaller change of probability, compared to the change for the same latent weight x in Gaussian/Laplace prior.

4.4.4 On the robustness of prior center vs prior spread against SEU

Based on the Table 4 above, we can also retrieve a conclusion about how the SEU interacts with the center prior parameter and spread prior parameter. For all metrics, **the center prior parameter is consistently more robust than the spread prior parameter**. Robustness of Bayesian Neural Networks against SEU injection on the prior center parameter comes from the nature of shifting the mean of the prior that moves all the weight samples by the same absolute amount, preserving the relative positions and allowing them to cancel out during the Monte Carlo averaging. Shift in the spread parameter will change the weight sampled to be significantly different from the mean (either pushing some further into the tails or other closer to the center), making the resulting logits to be much different than before the SEU injection happened.

4.5 Robustness by activation function

	AAAD		ASD		ARIn	
	DNN	BNN	DNN	BNN	DNN	BNN
R	.0078	.0168	.2651	.0958	.1875	.0688
W	.0178	.0213	.2716	.0964	.1924	.0698
ru	.0234	.0379	.2835	.1626	.2012	.1180
r6	.0114	.0248	.2653	.1012	.1878	.0737
sg	.0136	.0190	.2598	.0800	.1839	.0581
sn	.0141	.0294	.2702	.1056	.1913	.0775
th	.0163	.0214	.2714	.1102	.1923	.0794

Table 5: Robustness of Deterministic and Bayesian CNN model by activation function

For Table 5, we code each activation function as follows: RWG (R), WG (W), relu (ru), relu6 (r6), sigmoid (sg), sin (sn), tanh (th). We can observe that the Rectified Weighted Gaussian activation function has the most robust absolute accuracy difference among all activation function for the deterministic CNN model (which has been proven in (Dennis and Pope, 2025)) and Bayesian CNN Model. On

our newly introduced softmax difference measure, the sigmoid activation function achieves the lowest Average Softmax Difference (ASD) in both settings.

Relu, which is the most expressive activation function that we have in this experiment (as seen from Figure 4), shown a comparably better test accuracy despite showing the worst robustness. RWG, which yields the lowest test accuracy, achieves the strongest robustness, especially in terms of absolute accuracy difference. This observation shown us a symptom of a **trade-off between test accuracy and SEU robustness**, which is also observed in the work of (Carbone et al., 2020).

It can also be seen from Table 5 that the sigmoid activation function have the second best robustness in absolute accuracy difference on BNNs, while also having the best softmax difference. SEU happens to weight and bias before activation function is implemented. Let’s get back on how the neuron is calculated in the feed-forward process. This process, as mentioned in (Bishop, 2007) is notated as:

$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \quad (11)$$

where $x_1, x_2, \dots, x_D$ are inputs, $j = 1, 2, \dots, M$ and the superscript (1) represents the parameters in the first ‘layer’ of the network, which have a ‘width’ of M . $w_{j0}^{(1)}$ is Each of a_j is then transformed in the following equation:

$$z_j = h(a_j) \quad (12)$$

where $h(\cdot)$ is a differentiable activation function. In order to understand how different activation function can yield different robustness, let’s compare the sigmoid and tanh activation functions. Sigmoid has a smaller gradient $h'(\cdot)$ near zero (0.25) than tanh (1), meaning that weight perturbations around this region result in a smaller output change. This explains the sigmoid’s higher robustness under SEU injection.

4.6 Robustness by layer and layer parameter

Table 7 summarizes the SEU Injection result by averaging the value of Absolute Accuracy Difference and Softmax Difference across layers and layer parameters of the CNN architecture. Based on that table, it is observed that convolutional layer 2 is the most robust, followed by convolutional layer 1. The fully connected layer is the least robust, as it exhibits the highest mean of softmax difference and the highest mean absolute accuracy difference. These findings also align with the result shown in (Dennis and Pope,

		AAAD		ASD	
		DNN	BNN	DNN	BNN
conv1	bias	.0048	.0095	.2651	.0641
conv1	weight	.0070	.0129	.2696	.0762
conv2	bias	.0021	.0089	.2622	.0723
conv2	weight	.0058	.0136	.2640	.0987
fc1	bias	.0326	.0459	.2773	.1720
fc1	weight	.0372	.0555	.2792	.1611

Table 6: Robustness of Deterministic and Bayesian CNN model by layer and layer parameter (AAAD & ASD)

		ARIn	
		DNN	BNN
conv1	bias	.1875	.0458
conv1	weight	.1907	.0546
conv2	bias	.1854	.0515
conv2	weight	.1868	.0704
fc1	bias	.1974	.1259
fc1	weight	.1991	.1205

Table 7: Robustness of Deterministic and Bayesian CNN model by layer and layer parameter (ARIn)

2025), where the further convolutional layer (conv2 and conv3) have the lowest average test delta and the fully connected layer has the highest.

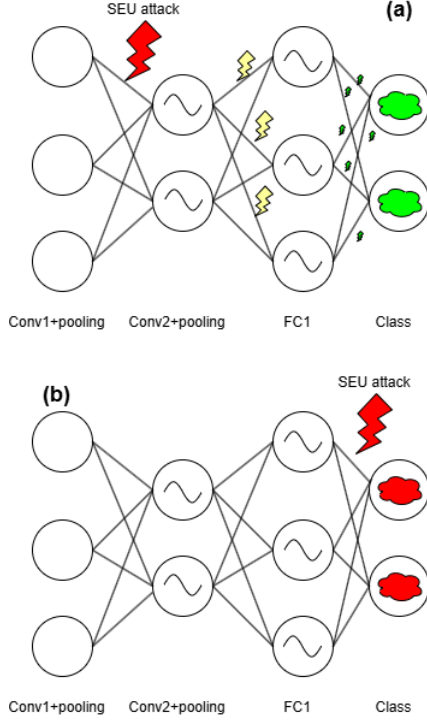
4.6.1 Layer SEU Robustness, propagation depth

The fully connected layer is the least robust because SEU injections directly affect the final logits without any subsequent non-linear transformations, resulting in the highest AAAD and ASD. In contrast, faults injected in earlier convolutional layers are partially mitigated by activation and pooling operations, which dampen or average out perturbations before they reach the output. As illustrated in Figure 6, early-layer faults weaken as they propagate through the network, whereas faults at the fully connected layer propagate directly to the output, causing the most severe distortions.

4.6.2 Layer component SEU Robustness, weight vs bias

Based on the result of our experiment, we can see that the weight is less robust compared to the bias, when we inject them with SEU/biflips. This is due to the nature of the neural networks itself. Looking back at Equation (11), we can see that the weight is multiplied by the input value x_i to compute a_j . **Change in weight to the value of a_j is amplified by the x_i** , compared to the change in bias that is merely a constant.

Figure 6: Illustration of SEU injection and its propagated effect on conv1 layer (a) and fc1 layer (b) in a CNN model



4.7 Robustness by attacked bit location

	ALAD		ARIn	
	DNN	BNN	DNN	BNN
0	-1.1804	-0.7909	.1848	.0381
1	37.0504	37.8829	.2371	.3009
3	-1.4815	-0.6454	.1847	.0376
6	-0.5849	-0.4291	.1850	.0378
10	-2.2308	-1.3945	.1847	.0374
15	-3.7360	-2.8997	.1847	.0374
21	-5.5422	-4.7058	.1847	.0376

Table 8: Robustness of Deterministic and Bayesian CNN model by attacked bit location (ALAD & ARIn)

To analyze the effect of SEU injections at different bit positions, we compute the log absolute difference of values before and after an SEU, defined as:

$$\text{Log Absolute Difference} = \log_{10}(|\text{value after SEU} - \text{value before SEU}|) \quad (13)$$

In this experiment, the attacked bit index in this experiment are sign (index 0), exponent (index 1,3,6), and mantissa (index 10, 15, 21). As shown in Table 9, flipping bit index 1 yields an average log absolute difference of approximately 37 and results in

	AAAD		ASD	
	DNN	BNN	DNN	BNN
0	.0005	.0075	.2613	.0533
1	.1023	.1206	.3193	.4080
3	.0002	.0070	.2612	.0527
6	.0013	.0070	.2617	.0529
10	.0001	.0067	.2611	.0525
15	.0000	.0067	.2611	.0525
21	.0000	.0067	.2611	.0528

Table 9: Robustness of Deterministic and Bayesian CNN model by attacked bit location (AAAD & ASD)

the worst robustness across all metrics (AAAD, ASD, and ARIn). By contrast, perturbations to the mantissa bits have negligible impact on AAAD. Attacks on bit index 6 yield slightly higher AAAD than the mantissa bits, through still minor compared to bit index 1.

Notably, no clear correlation can be established between the log absolute difference and robustness metrics such as AAAD or ASD. Bit index 0 produces a moderate log absolute difference yet yields slightly higher ASD compared to mantissa flips.

4.8 Robustness by original bit value

	AAAD		ASD		ARIn	
	DNN	BNN	DNN	BNN	DNN	BNN
0	.0275	.0455	.2767	.1734	.1966	.1268
1	.0002	.0069	.2612	.0529	.1847	.0378

Table 10: Robustness of Deterministic and Bayesian CNN model by original bit attacked

In this subsection, we also compare how the robustness is affected by the original bit value/state before the SEU. Table 10 shows that bitflip from 1 to 0 exhibits an extremely more robust model compared to bitflip from 0 to 1, especially on the absolute accuracy difference. It's not aligned with the claim from (Hanif et al., 2021) that stated the opposite.

This difference can be explained by the distribution of weights and biases in our models, which are typically close to 0 or 1. Consequently, the original bit value/state play an important role in determining the impact of SEUs. After a thorough observation, we saw that in all SEU injections involving bit index 1, the original bit value/state was always 0.

5 Conclusion

This research explored the performance of Bayesian Neural Networks (BNNs) robustness against Single Event Upsets (SEUs). Our initial findings show that the Deterministic CNN model is better in test accuracy compared to the Bayesian CNN model. On the other hand, Bayesian CNN model is more robust, looking at its performance in ARIn and Softmax Difference. Based on our experiment result, model with dropout is the most robust for Bayesian CNN. The Gaussian prior is the most robust. SEU against the prior center parameter is more robust than the spread parameter. Sigmoid offers better robustness in ASD and ARIn, RWG is the most robust in AAAD. Trade-off between test accuracy and robustness is observed. Convolutional layers are more robust than the fully connected layer. Attack on the first exponent bit is the most devastating; mantissa bit flips had a minimum effect. Flipping a bit from 1 to 0 is found to be far more robust.

REFERENCES

- Australian Transport Safety Bureau (2009). Interim factual report no. 2: In-flight upset involving airbus a330-303, vh-qpa, 154 km west of learnmonth, wa on 7 october 2008. Technical report, Australian Transport Safety Bureau, Canberra, ACT, Australia. Aviation Occurrence Investigation AO-2008-070.
- Barber, D. and Bishop, C. (1998). Ensemble learning in bayesian neural networks. In *Generalization in Neural Networks and Machine Learning*, pages 215–237, United Kingdom: Springer.
- Bishop, C. M. (2007). *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer, 1 edition.
- Blaas, A. (2021). *On the adversarial robustness of Bayesian machine learning models*. PhD thesis, University of Oxford. PhD thesis.
- Blundell, C., Cornebise, J., Kavukcuoglu, K., and Wierstra, D. (2015). Weight uncertainty in neural networks. In *Proceedings of the 32nd International Conference on Machine Learning - Volume 37, ICML'15*, page 1613–1622. JMLR.org.
- Bosio, A., Bernardi, P., Ruospo, A., and Sanchez, E. (2019). A reliability analysis of a deep neural network. In *2019 IEEE Latin American Test Symposium (LATS)*, pages 1–6.
- Carbone, G., Wicker, M., Laurenti, L., Patane, A., Bortolussi, L., and Sanguinetti, G. (2020). Robustness of bayesian neural networks to gradient-based attacks. *CoRR*, abs/2002.04359.
- Chen, X., Liu, C., Zhao, Y., Jia, Z., and Jin, G. (2022). Improving adversarial robustness of bayesian neural networks via multi-task adversarial training. *Information Sciences*, 592:156–173.
- Dennis, W. and Pope, J. (2025). A framework for developing robust machine learning models in harsh environments: A review of cnn design choices. In *Proceedings of the 17th International Conference on Agents and Artificial Intelligence - Volume 2: ICAART*, pages 322–333. INSTICC, SciTePress.
- Feng, Y., Rudner, T. G. J., Tsilivis, N., and Kempe, J. (2024). Attacking bayes: On the adversarial robustness of bayesian neural networks.
- Ferrante, M., Boccato, T., and Toschi, N. (2024). Enabling uncertainty estimation in neural networks through weight perturbation for improved alzheimer’s disease classification. *Frontiers in Neuroinformatics*, Volume 18 - 2024.
- Giuffrida, G., Diana, L., de Gioia, F., Benelli, G., Meoni, G., Donati, M., and Fanucci, L. (2020). CloudScout: A deep neural network for on-board cloud detection on hyperspectral images. *Remote Sens. (Basel)*, 12(14):2205.
- Hammell, R. (2018). Ships in satellite imagery. <https://www.kaggle.com/dsv/61115>.
- Hanif, M. A., Khalid, F., Putra, R. V. W., Teimoori, M. T., Kriebel, F., Zhang, J. J., Liu, K., Rehman, S., Theocharides, T., Artusi, A., Garg, S., and Shafique, M. (2021). *Robust Computing for Machine Learning-Based Systems*, pages 479–503. Springer International Publishing, Cham.
- Hari, S. K. S., Tsai, T., Stephenson, M., Keckler, S. W., and Emer, J. (2017). SASSIFI: An architecture-level fault injection tool for gpu application resilience evaluation. In *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 249–258. IEEE.
- IEEE Computer Society (2019). IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2019. Published on 2019-07-22.
- MacKay, D. J. C. (1992). A practical bayesian framework for backpropagation networks. *Neural Comput.*, 4(3):448–472.
- Neal, R. M. (1996). *Bayesian Learning for Neural Networks*. Springer-Verlag, Berlin, Heidelberg.
- Pang, Y., Cheng, S., Hu, J., and Liu, Y. (2021). Evaluating the robustness of bayesian neural networks against different types of attacks.
- Petersen, E. (2011). *Single Event Effects in Aerospace*. Wiley.
- Santos, F. F. d., Pimenta, P. F., Lunardi, C., Draghetti, L., Carro, L., Kaeli, D., and Rech, P. (2019). Analyzing and increasing the reliability of convolutional neural networks on gpus. *IEEE Transactions on Reliability*, 68(2):663–677.
- Shafique, M., Naseer, M., Theocharides, T., Kyrkou, C., Mutlu, O., Orosa, L., and Choi, J. (2021). Robust machine learning systems: Challenges, current trends, perspectives, and the road ahead. *CoRR*, abs/2101.02559.
- Su, D., Zhang, H., Chen, H., Yi, J., Chen, P., and Gao, Y. (2018). Is robustness the cost of accuracy? - A com-

prehensive study on the robustness of 18 deep image classification models. *CoRR*, abs/1808.01688.

- Yan, Z., Shi, Y., Liao, W., Hashimoto, M., Zhou, X., and Zhuo, C. (2020). When single event upset meets deep neural networks: Observations, explorations, and remedies. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 163–168.
- Zhang, J., Hua, Y., Xue, Z., Song, T., Zheng, C., Ma, R., and Guan, H. (2021). Robust bayesian neural networks by spectral expectation bound regularization. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3814–3823.